

II- Sem MCA –C
M23DE0206 - Machine Learning Lab using Python
List of Laboratory Programs

Part A

(Minimum 80% of programs are mandatory)

**Sl.
No.**

1 Write a program to implement how to read and display a dataset in Python.

```
# Import necessary libraries
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
# 1. from Google drive
from google.colab import drive
drive.mount('/content/drive')
df=pd.read_csv("/content/drive/MyDrive/breast-cancer.csv")
#df = pd.read_csv("/content/sample_data/california_housing_train.csv")

# 2. from url
url = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv'
#df = pd.read_csv(url)

# 3. from sklearn datasets
#from sklearn.datasets import load_breast_cancer
#cancer = load_breast_cancer()
#df=pd.DataFrame(cancer.data, columns= cancer.feature_names)
#from sklearn.datasets import load_iris
#iris = load_iris()
#df=pd.DataFrame(iris.data, columns= iris.feature_names)

# 4. from local file
#from google.colab import files
#uploaded = files.upload()
#df = pd.read_csv("Toddler_V_2.csv")

# Display basic properties of the dataset
print("Dataset Head:")
print(df.head()) # Display the first few rows of the dataset
```

```

print("\nDataset Info:")
print(df.info()) # Display summary of the dataset including column data types and non-null counts

print("\nDataset Description:")
print(df.describe()) # Display statistical summary of numeric columns

print("\nDataset Shape:")
print(df.shape) # Display the dimensions of the dataset (rows, columns)

print("\nColumn Names:")
print(df.columns) # Display column names

print("\nMissing Values:")
print(df.isnull().sum()) # Display the number of missing values per column

# Optional: Plotting some basic visualizations
print("\nBasic Data Visualization:")
df.hist(figsize=(10, 8)) # Plot histograms for numeric columns
plt.show()

```

2 **Write a program to learn how to select features for machine learning**

```

import numpy as np
import pandas as pd
from sklearn.feature_selection import SelectKBest, f_classif, RFE
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Load dataset
from sklearn.datasets import load_iris
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 1. Univariate Feature Selection
print("Univariate Feature Selection:")
# Apply SelectKBest
selector = SelectKBest(score_func=f_classif, k=2)
selector.fit(X, y)
# Get the scores and p-values for each feature

```

```

scores = selector.scores_
p_values = selector.pvalues_
# Create a DataFrame to display the results
feature_scores_df = pd.DataFrame({
    'Feature': feature_names,
    'Score': scores,
    'P-value': p_values
}).sort_values(by='Score', ascending=False)
print("\nFeature scores:")
print(feature_scores_df)

#Display the selected features
selected_features = np.array(feature_names)[selector.get_support()]
print("\nSelected features:")
print(selected_features)

# 2. Recursive Feature Elimination (RFE)
# Train a classifier
clf = RandomForestClassifier()
model = clf.fit(X_train, y_train)
print("\nRecursive Feature Elimination (RFE):")
rfe = RFE(estimator=model, n_features_to_select=2)
rfe.fit(X_train, y_train)
scores = rfe.ranking_
feature_rfescores_df = pd.DataFrame({'Feature': feature_names, 'Score':
scores}).sort_values(by='Score', ascending=False)
print("\nFeature rfescores:")
print(feature_rfescores_df)

# Display selected features
print("Selected features:", X_train.columns[rfe.support_])

# 3. Feature Importance from Random Forest
print("\nFeature Importance from Random Forest:")
# Display feature importance
importances = model.feature_importances_
indices = np.argsort(importances)[::-1]
print("Feature ranking:")
for f in range(X_train.shape[1]):
    print(f"{X_train.columns[indices[f]]}: {importances[indices[f]]}")

```